INSTITUT SUPERIEUR D'INFORMATIQUE

ET DES TECHNIQUES DE COMMUNICATION – HAMMAM SOUSSE

المعهد العالي للإعلامية وتكنولوجيات الاتصال بسوسة

End-of-Studies Internship Report

Presented for obtaining the Bachelor degree in Computer Science
Spécialité : Informatique and Multimédia

CyberScope

Web Application Security Auditing System
Detecting and Analyzing Vulnerabilities
Through Intelligent Automation

ELABORATED BY
MOHAMED GUIZANI

Academic Supervisor: M.Mohamed Amine Ben Amar

Professional Supervisor: M.Yassine Lasoued

Host Company

EY

School Year : 2025/2026

Higher Institute of Computer Science and Communication Technologies ISITCOM Tel/Fax: +216 73 37 15 71 / +216 73 36 44 11

Dedications

First and foremost, I thank **Allah** for choosing me among thousands to face this trial, which has made me more patient, wiser, and stronger. Despite the back injury that has challenged me for more than a year and a half, during which I could barely sit to work, all praise be to Allah for His guidance and strength.

To my beloved family, who replaced my physical pain with their unwavering support, love, and care:

To **Anissa, my mother**, whose devotion knew no bounds—checking on my condition and illness every single day throughout this difficult period. Your constant care has been my healing.

To **Hassen, my father**, who dedicated himself entirely to ensuring my comfort, doing everything possible to ease my burden. Your sacrifices have not gone unnoticed.

To **Iskander**, who became my backbone and eternal supporter when my own failed me. You carried me through the darkest moments.

To **Sabri**, from whom I drew power and strength when mine was depleted. Your presence gave me energy to continue.

To **Naktel**, the youngest among us, who brought joy and happiness into days filled with pain. Your light brightened my path.

Every line of this report was written through immense pain and is dedicated to all of you. Your love transformed my struggle into strength, and your faith in me made this achievement possible.

Alhamdulillahi rabbil alameen.

Acknowledgments

I would like to express my deepest gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I extend my heartfelt appreciation to **M. Yassine Lassoed**, my professional supervisor, who believed in me and in this project from the very beginning. Despite the challenges faced during this internship, he provided me with essential guidance and constant support, enabling me to bring this project to life. His encouragement and trust in my abilities have been invaluable throughout this journey.

I would also like to sincerely thank **M. Mohamed Amine Ben Amar**, my academic supervisor, for his continuous support and valuable advice. Throughout this challenging period, he offered insightful feedback, motivation, and understanding, which greatly contributed to the successful completion of this work.

I am equally grateful to all the individuals who supported me, directly or indirectly, during the realization of this project. Their encouragement and assistance played a significant role in achieving this accomplishment.

Thank you for making this achievement possible.

Table of Contents

Dedications **i**

Acknowledgements **ii**

Table of Contents **iii**

List of Figures **vi**

List of Tables **vii**

General Introduction **1**

1 General Presentation of the Project **2**

1.1 Introduction 2

1.2 Presentation of the Host Organization 2

1.3 Project Context and Problem Statement 3

1.4 Overview of Web Application Security 3

1.4.1 OWASP Top 10 and Modern Threat Landscape 4

1.4.2 Limitations of Traditional Vulnerability Scanners 4

1.5 Study of Existing Solutions 5

1.5.1 Nikto 5

1.5.2 OWASP ZAP 5

1.5.3 Burp Suite 5

1.5.4 Comparative Analysis 6

1.6 Critique of Existing Solutions 7

1.7 Proposed Solution: CyberScope 7

1.8 Adopted Development Methodology 8

1.9 Conclusion 8

2 System Analysis and Requirements Specification **9**

2.1 Introduction 9

2.2 Stakeholder Identification 9

2.3 System Overview 9

2.4 Functional Requirements 9

2.4.1 Scan Initiation and Target Submission 9
2.4.2 Intelligent Tool Orchestration 9
2.4.3 Dynamic Application Crawling (BrowserAgent) 9
2.4.4 Vulnerability Detection and Analysis 9
2.4.5 Results Aggregation and Visualization 9
2.5 Non-Functional Requirements 10
2.5.1 Performance and Scalability 10
2.5.2 Security Requirements 10
2.5.3 Usability 10
2.6 Use Case Diagram 10
2.7 Conclusion 10

3 System Design and Architecture 11
3.1 Introduction 11
3.2 Global System Architecture 11
3.2.1 AI-Driven Offensive Intelligence Core 11
3.2.2 Orchestration Backend (Node.js) 11
3.2.3 Offensive Dashboard (React) 11
3.3 Detailed Component Design 11
3.3.1 BrowserAgent and Native SPA Fuzzer 11
3.3.2 Vulnerability Analyzer and Correlation Engine 11
3.3.3 Scan Lifecycle Management 11
3.4 Intelligence Pipeline 12
3.4.1 Reconnaissance and Target Fingerprinting 12
3.4.2 Autonomous Tool Selection 12
3.4.3 Cross-Tool Normalization 12
3.4.4 Attack Path Visualization 12
3.4.5 Intelligent Findings Contextualization (LLMs) 12
3.5 UML Modeling 12
3.5.1 Use Case Diagrams 12
3.5.2 Sequence Diagrams 12
3.5.3 Class Diagram 12
3.6 Conclusion 12

4 Implementation and Validation 13
4.1 Introduction 13
4.2 Development Environment 13
4.2.1 Technologies and Frameworks 13

4.2.2 Tools and Security Modules 13
4.3 System Implementation 13
4.3.1 AI Core Integration 13
4.3.2 Backend Orchestration (Node.js/Express) 13
4.3.3 Offensive Dashboard: Real-time Streaming 13
4.4 Experimental Validation 13
4.4.1 Test Environment and Targets 13
4.4.2 Case Study: OWASP Juice Shop 13
4.4.3 Case Study: Altoro Mutual 14
4.4.4 Performance and Accuracy Evaluation 14
4.5 Discussion of Results 14
4.5.1 True Positives vs False Positives 14
4.5.2 Missed Vulnerabilities Analysis 14
4.5.3 System Limitations 14
4.6 Conclusion 14

5 Results, Improvements and Future Work 15
5.1 Introduction 15
5.2 Comparative Evaluation with Existing Tools 15
5.3 Identified Limitations 15
5.4 Proposed Enhancements 15
5.5 Future Research Directions 15
5.6 Conclusion 15

References 16

General Conclusion 18

List of Figures

1.1 EY Tunisia 3
1.2 Nikto 5
1.3 OWASP ZAP 5
1.4 Burp Suite 6

List of Tables

1.1 Comparative analysis of existing solutions 6
1.2 Synthesis of limitations of existing solutions 7

General Introduction

In recent years, the rapid evolution of web technologies has significantly transformed the development and deployment of modern applications. Web applications have become increasingly dynamic, interactive, and complex, relying heavily on client-side rendering and continuous communication with backend services. While these advancements have enhanced user experience, they have also expanded the attack surface and introduced new security challenges.

Web application security has become a critical concern for organizations. Vulnerabilities such as SQL injection, cross-site scripting (XSS), and authentication flaws remain among the most exploited attack vectors, as highlighted by the OWASP Top 10. These vulnerabilities can lead to severe consequences, including data breaches, financial losses, and reputational damage.

To mitigate these risks, various security scanning tools have been developed, including solutions such as Nikto, OWASP ZAP, and Burp Suite. Although these tools provide valuable capabilities, they exhibit significant limitations. In particular, traditional scanners struggle to effectively analyze modern Single Page Applications (SPAs), where content is dynamically generated through JavaScript. Furthermore, they often produce redundant or low-priority findings, making vulnerability analysis more complex and less efficient.

In this context, there is a growing need for more intelligent and adaptive security auditing systems capable of handling modern web architectures while improving detection accuracy and result relevance.

To address these challenges, this project introduces **CyberScope**, an automated web application security auditing system designed to detect and analyze vulnerabilities through intelligent orchestration and advanced automation techniques. The proposed system integrates multiple security tools within a unified pipeline and enhances their capabilities through a browser-based crawling agent capable of exploring dynamic applications. In addition, an AI-driven correlation engine is employed to normalize, deduplicate, and prioritize detected vulnerabilities, providing more meaningful and actionable insights.

The objective of this work is to design and implement a comprehensive and efficient security auditing platform that improves vulnerability detection coverage and adapts to the complexity of modern web environments.

Chapter 1

General Presentation of the Project

1.1 Introduction

This chapter focuses on the general context of the project. It presents the background of the work, the security challenges faced by modern web applications, the limitations of existing solutions, and the motivation behind the proposed system, **CyberScope**.

1.2 Presentation of the Host Organization

Ernst & Young (EY) [1] is one of the world's leading professional services firms, recognized globally for its expertise in assurance, consulting, strategy, tax, and transaction services. Founded in 1989 through the merger of Ernst & Whinney and Arthur Young, EY operates in more than 150 countries and employs hundreds of thousands of professionals worldwide.

In Tunisia, EY plays a key role in supporting organizations in their digital transformation and technological innovation. The firm assists both public and private institutions in addressing complex business challenges by providing tailored consulting services and advanced technical solutions.

EY Tunisia offers a wide range of services, including IT consulting, cybersecurity, risk management, and digital innovation. In particular, the firm contributes to strengthening information systems security by identifying vulnerabilities and implementing robust protection mechanisms.

Through its global network and local expertise, EY promotes best practices in cybersecurity and technological development. Its commitment to quality and innovation makes it an ideal environment for the development of advanced solutions such as automated web security auditing systems.

Figure 1.1: EY Tunisia

1.3 Project Context and Problem Statement

With the increasing reliance on web applications in critical domains such as finance, e-commerce, and healthcare, ensuring their security has become a major challenge. Modern applications are more dynamic and complex, which significantly increases their exposure to various cyber threats such as SQL injection, cross-site scripting, and authentication vulnerabilities.

Despite the availability of several security tools, existing solutions often fail to effectively analyze modern web applications, especially those based on dynamic architectures. These tools may miss critical vulnerabilities, generate redundant results, or require significant manual intervention, which reduces their efficiency and reliability.

In this context, the main problem addressed in this project is the lack of an intelligent and automated solution capable of efficiently auditing modern web applications while improving vulnerability detection accuracy and reducing false positives.

The objective of this project is therefore to design and develop **CyberScope**, an advanced web application security auditing system that enhances vulnerability detection through intelligent orchestration, automation, and improved analysis of security findings.

1.4 Overview of Web Application Security

Web application security refers to the set of measures and practices implemented to protect web applications from vulnerabilities and cyber threats. As web applications become increasingly central to business operations and data management, ensuring their security is essential to preserve the confidentiality, integrity, and availability of information.

Modern web applications are exposed to a wide range of attacks that exploit weaknesses in their design, implementation, or configuration. These attacks may target input validation, authentication mechanisms, session management, or communication

between client and server. As a result, a single vulnerability can compromise the entire system and lead to severe consequences such as data breaches, financial loss, or reputational damage.

To mitigate these risks, security must be integrated throughout the entire development lifecycle. This includes secure coding practices, regular vulnerability assessments, and the use of automated security tools capable of identifying potential weaknesses before they are exploited.

In this context, standards such as the OWASP Top 10 provide valuable guidance by identifying the most critical web application security risks [2]. These references help developers and security professionals prioritize their efforts and implement effective protection strategies.

1.4.1 OWASP Top 10 and Modern Threat Landscape

The OWASP Top 10 is a widely recognized standard that identifies the most critical security risks affecting web applications [2]. It provides a prioritized list of vulnerabilities such as injection attacks, broken authentication, sensitive data exposure, and cross-site scripting.

These vulnerabilities represent the most common attack vectors exploited by attackers in real-world scenarios. As web technologies evolve, the threat landscape continues to expand, with modern applications introducing new risks related to client-side execution, APIs, and complex system interactions.

By highlighting the most significant threats, the OWASP Top 10 helps developers and security professionals focus on the most impactful vulnerabilities and adopt effective mitigation strategies.

1.4.2 Limitations of Traditional Vulnerability Scanners

Traditional vulnerability scanners are widely used to detect security flaws in web applications through automated analysis and predefined signatures. While these tools are effective for identifying known vulnerabilities, they present several limitations when applied to modern web environments.

In particular, they often struggle to analyze dynamic applications, especially those based on client-side rendering and asynchronous communication. As a result, important attack surfaces may remain unexplored. Additionally, these tools may generate a high number of false positives and redundant findings, which complicates the analysis process and reduces their overall efficiency.

These limitations highlight the need for more advanced and intelligent approaches to web application security auditing.

1.5 Study of Existing Solutions

Several tools have been developed to identify vulnerabilities in web applications and assist security professionals in detecting potential threats. These tools provide different approaches to vulnerability scanning, ranging from simple automated checks to advanced interactive analysis. Among the most widely used solutions are Nikto, OWASP ZAP, and Burp Suite.

1.5.1 Nikto

Nikto is an open-source web server scanner designed to detect known vulnerabilities, outdated software, and misconfigurations in web servers [4]. It performs comprehensive checks against a large database of known issues and is commonly used for quick security assessments.

However, Nikto is limited to server-side analysis and does not effectively handle modern dynamic web applications, which reduces its applicability in complex environments.

Figure 1.2: Nikto

1.5.2 OWASP ZAP

OWASP ZAP is a widely used open-source security testing tool that provides automated scanning as well as manual testing capabilities [3]. It is capable of detecting a variety of vulnerabilities such as injection flaws and security misconfigurations.

Despite its flexibility, ZAP may require proper configuration and user expertise to achieve optimal results, and it can produce a significant number of false positives in complex applications.

Figure 1.3: OWASP ZAP

1.5.3 Burp Suite

Burp Suite is a professional web security testing platform offering a comprehensive set of tools for vulnerability detection and exploitation [5]. It allows detailed

analysis of HTTP requests and responses and supports advanced manual testing techniques.

While powerful, Burp Suite is more complex to use and often requires significant manual intervention, which can make the testing process time-consuming.

Figure 1.4: Burp Suite

1.5.4 Comparative Analysis

To better evaluate the strengths and limitations of the studied tools, a comparative analysis is presented below. Although Nikto, OWASP ZAP, and Burp Suite are widely used in web application security testing, each of them addresses only part of the overall auditing process.

Nikto is well suited for rapid server-side checks, OWASP ZAP provides a balanced approach between automation and usability, and Burp Suite offers advanced manual testing capabilities. However, none of these tools fully combines intelligent automation, deep dynamic exploration, and effective reduction of noisy findings.

Table 1.1: Comparative analysis of existing solutions

Criteria	**Nikto**	**OWASP ZAP**	**Burp Suite**
Type	Open-source server scanner	Open-source proxy and scanner	Professional security testing suite
Main approach	Signature-based scanning	Automated and manual testing	Advanced manual and automated testing
Dynamic application support	Limited	Partial	Good
Ease of use	Easy	Moderate	Complex
False positives	High	Medium	Low to medium
Manual testing support	No	Yes	Advanced
Automation level	High	High	Medium
Best suited for	Quick server checks	General web security testing	Deep vulnerability analysis

This comparison shows that existing tools provide useful but incomplete capabilities. Their limitations in terms of automation, adaptability, and dynamic analysis

justify the need for a more integrated and intelligent solution such as **CyberScope**.

1.6 Critique of Existing Solutions

Although existing tools such as Nikto, OWASP ZAP, and Burp Suite provide valuable support for vulnerability detection, they still present several limitations when applied to modern web applications.

These limitations are mainly related to the lack of support for dynamic applications, the generation of false positives, and the need for manual intervention. In addition, the absence of intelligent correlation between detected vulnerabilities reduces the clarity and efficiency of the analysis process.

Table 1.2: Synthesis of limitations of existing solutions

Limitation	**Description**	**Impact**
Limited support for dynamic applications	Difficulty handling JavaScript-based applications	Incomplete vulnerability detection
High false positives	Redundant or irrelevant findings	Time-consuming analysis
Lack of intelligent correlation	No prioritization of vulnerabilities	Reduced clarity of results
Manual intervention required	Dependence on user expertise	Low scalability
Fragmented tool usage	Multiple tools used separately	Complex workflow

These limitations highlight the need for a more advanced solution capable of combining automation, accuracy, and adaptability.

1.7 Proposed Solution: CyberScope

To overcome the limitations identified in existing solutions, this project proposes **CyberScope**, an intelligent web application security auditing system.

CyberScope is based on a unified architecture that integrates multiple security tools into a single automated pipeline. It introduces a browser-based crawling mechanism capable of exploring modern dynamic applications and interacting with client-side components.

In addition, the system incorporates an intelligent analysis layer responsible for normalizing, deduplicating, and prioritizing vulnerabilities. This significantly reduces noise and improves the relevance of the results.

By combining automation, dynamic exploration, and intelligent processing, Cy-

berScope aims to provide a more efficient and scalable approach to web application security auditing.

1.8 Adopted Development Methodology

To ensure an efficient and flexible development process, the Scrum methodology was adopted for the realization of this project. Scrum is an agile framework that promotes iterative development, continuous feedback, and adaptability.

The project was organized into multiple sprints, each focusing on specific objectives such as backend development, tool integration, frontend implementation, and system refinement. This iterative approach allowed progressive improvement of the system while ensuring regular validation of functionalities.

Throughout the development process, tasks were planned, executed, and evaluated in each sprint. This facilitated rapid issue resolution, continuous enhancement, and better management of project complexity.

The adoption of Scrum contributed to improving productivity, organization, and adaptability, which were essential for the successful development of **CyberScope**.

1.9 Conclusion

This chapter introduced the general context of the project by highlighting the importance of web application security and the challenges posed by modern dynamic applications. It also presented a study of existing solutions and identified their main limitations.

Based on this analysis, the need for a more advanced and intelligent approach was established, leading to the proposal of **CyberScope**. The next chapters will focus on the system analysis, design, and implementation of the proposed solution.

Chapter 2

System Analysis and Requirements Specification

2.1 Introduction

This chapter presents the analysis of the proposed system, **CyberScope**. It identifies the main stakeholders involved in the platform and defines the functional and non-functional requirements that guide its development.

It also provides an overview of the system and highlights its core functionalities, including intelligent tool orchestration, dynamic application crawling, and automated vulnerability analysis, which form the foundation for the system design presented in the next chapter.

2.2 Stakeholder Identification

Content will be added here.

2.3 System Overview

Content will be added here.

2.4 Functional Requirements

Content will be added here.

2.4.1 Scan Initiation and Target Submission

Content will be added here.

2.4.2 Intelligent Tool Orchestration

Content will be added here.

2.4.3 Dynamic Application Crawling (BrowserAgent)

Content will be added here.

2.4.4 Vulnerability Detection and Analysis

Content will be added here.

2.4.5 Results Aggregation and Visualization

Content will be added here.

2.5 Non-Functional Requirements

Content will be added here.

2.5.1 Performance and Scalability

Content will be added here.

2.5.2 Security Requirements

Content will be added here.

2.5.3 Usability

Content will be added here.

2.6 Use Case Diagram

Content will be added here.

2.7 Conclusion

Content will be added here.

Chapter 3

System Design and Architecture

3.1 Introduction

This chapter presents the design and architecture of the proposed solution. It describes the global structure of the system, the detailed design of its components, the intelligence pipeline, and the UML models used to formalize the solution.

3.2 Global System Architecture

Content will be added here.

3.2.1 AI-Driven Offensive Intelligence Core

Content will be added here.

3.2.2 Orchestration Backend (Node.js)

Content will be added here.

3.2.3 Offensive Dashboard (React)

Content will be added here.

3.3 Detailed Component Design

Content will be added here.

3.3.1 BrowserAgent and Native SPA Fuzzer

Content will be added here.

Request Interception and JS Hooking

Content will be added here.

Mutation Algorithms for SQLi and IDOR

Content will be added here.

3.3.2 Vulnerability Analyzer and Correlation Engine

Content will be added here.

3.3.3 Scan Lifecycle Management

Content will be added here.

3.4 Intelligence Pipeline

Content will be added here.

3.4.1 Reconnaissance and Target Fingerprinting

Content will be added here.

3.4.2 Autonomous Tool Selection

Content will be added here.

3.4.3 Cross-Tool Normalization

Content will be added here.

3.4.4 Attack Path Visualization

Content will be added here.

3.4.5 Intelligent Findings Contextualization (LLMs)

Content will be added here.

3.5 UML Modeling

Content will be added here.

3.5.1 Use Case Diagrams

Content will be added here.

3.5.2 Sequence Diagrams

Content will be added here.

3.5.3 Class Diagram

Content will be added here.

3.6 Conclusion

This chapter described the design choices and architectural organization of the CyberScope system.

Chapter 4

Implementation and Validation

4.1 Introduction

This chapter describes the implementation and validation of CyberScope. It presents the development environment, the main implementation choices, the experimental setup, and the analysis of the obtained results.

4.2 Development Environment

Content will be added here.

4.2.1 Technologies and Frameworks

Content will be added here.

4.2.2 Tools and Security Modules

Content will be added here.

4.3 System Implementation

Content will be added here.

4.3.1 AI Core Integration

Content will be added here.

4.3.2 Backend Orchestration (Node.js/Express)

Content will be added here.

4.3.3 Offensive Dashboard: Real-time Streaming

Content will be added here.

4.4 Experimental Validation

Content will be added here.

4.4.1 Test Environment and Targets

Content will be added here.

4.4.2 Case Study: OWASP Juice Shop

Content will be added here.

4.4.3 Case Study: Altoro Mutual

Content will be added here.

4.4.4 Performance and Accuracy Evaluation

Content will be added here.

4.5 Discussion of Results

Content will be added here.

4.5.1 True Positives vs False Positives

Content will be added here.

4.5.2 Missed Vulnerabilities Analysis

Content will be added here.

4.5.3 System Limitations

Content will be added here.

4.6 Conclusion

This chapter presented the implementation of the system and discussed the main validation results.

Chapter 5

Results, Improvements and Future Work

5.1 Introduction

This chapter summarizes the final results of the project and discusses the main limitations, possible improvements, and future research directions for CyberScope.

5.2 Comparative Evaluation with Existing Tools

Content will be added here.

5.3 Identified Limitations

Content will be added here.

5.4 Proposed Enhancements

Content will be added here.

5.5 Future Research Directions

Content will be added here.

5.6 Conclusion

This chapter highlighted the final outcomes of the project and outlined possible future evolutions of the proposed system.

References

Bibliography

[1] Ernst & Young (EY), *EY Tunisia Official Website*. Available at: https://www.ey.com/fr_ca/locations/tunisia

[2] OWASP Foundation, *OWASP Top 10 - Web Application Security Risks*, 2021.

[3] OWASP ZAP, https://www.zaproxy.org/

[4] Nikto Web Server Scanner, https://cirt.net/Nikto2

[5] PortSwigger, *Burp Suite*, https://portswigger.net

General Conclusion

Conclusion